

인구통계 대시보드 프로젝트 교육 가이드

01_population 인구통계 분석 시스템

작성일: 2026-01-11

대상: 신규 담당자 및 개발자

인구통계 대시보드 프로젝트 교육 가이드

작성일: 2026-01-11 프로젝트명: 01_population (인구통계 분석 시스템) 대상: 신규 담당자 및 개발자

목차

1. 프로젝트 개요 2. 데이터 설명 3. 시스템 아키텍처 플로우 4. 핵심 코드 설명 5. 지금까지 구축한 내용 6. 정기 운영 작업 7. Git 및 Cloudtype 배포 8. 문제 해결 가이드

1. 프로젝트 개요

1.1 시스템 소개

행정안전부 주민등록인구통계 API를 활용한 인구통계 분석 시스템입니다.

1.2 핵심 기능 3가지

인구통계 분석 시스템			
1. 데이터 수집	2. 대시보드	3. Text-to-SQL 챗봇	
공공API → DB	시각화 차트	자연어 질문 → SQL → 분석	

1.3 기술 스택

분류	기술
Backend	Python 3, Flask
Database	PostgreSQL (파티셔닝)
LLM	Claude, OpenAI GPT-4o, Solar
시각화	matplotlib, koreanize-matplotl
패키지관리	uv

2. 데이터 설명

2.1 데이터 출처

- 행정안전부 주민등록인구통계 (공공데이터포털) - 매월 말 기준 인구 데이터 제공 - 읍면동 단위까지 상세 데이터

2.2 수집 데이터 종류

수집 데이터 (4종)			
1. 인구 및 세대현황 (기본 통계) 총인구, 세대수		2. 연령대별 인구 (10세/5세 단위) 연령그룹별 남녀인구	
3. 1세별 인구 (0~110세+) ★ 가장 상세		4. 1세별 1인가구 (0~110세+) 1인가구 상세분석	

2.3 데이터베이스 테이블 구조

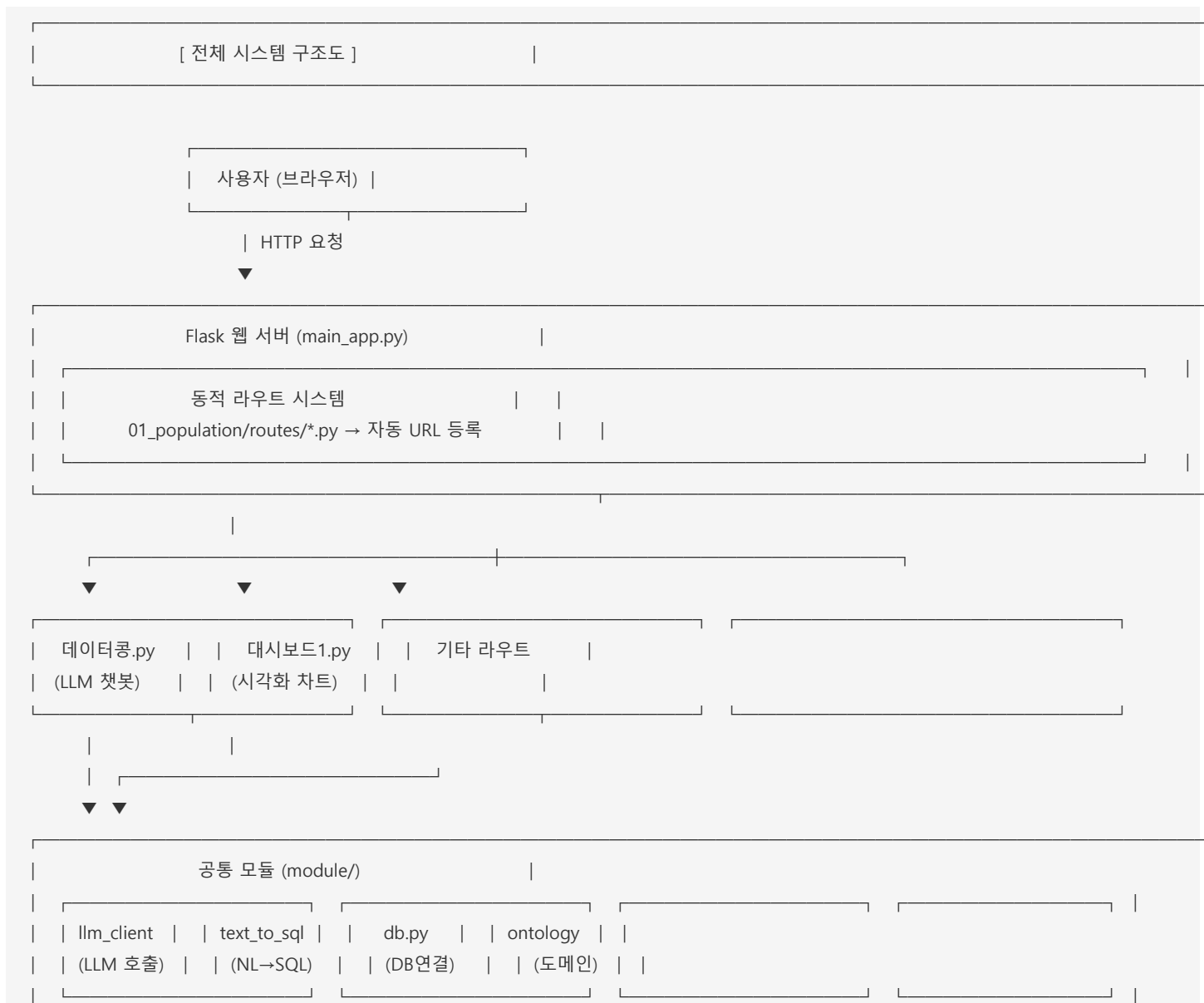
PostgreSQL Database			
[Dimension Table - 행정구역]			
dim_admin_area			
- admin_code (PK): 10자리 행정구역코드			
- sido_nm: 시도명 (예: 경상북도)			
- sigungu_nm: 시군구명 (예: 안동시)			
- eupmyeondong_nm: 읍면동명			
[Fact Tables - 인구 데이터] (파티셔닝 적용)			
fact_population_basic : 기본 인구통계			
fact_population_by_age : 1세별 인구 (컬럼 230개)			
fact_single_household : 1세별 1인가구			
[Cache Tables - 집계 캐시]			
cache_sigungu_indicators : 고령화율, 유소년율 등			
cache_sigungu_age_summary: 연령대별 요약			

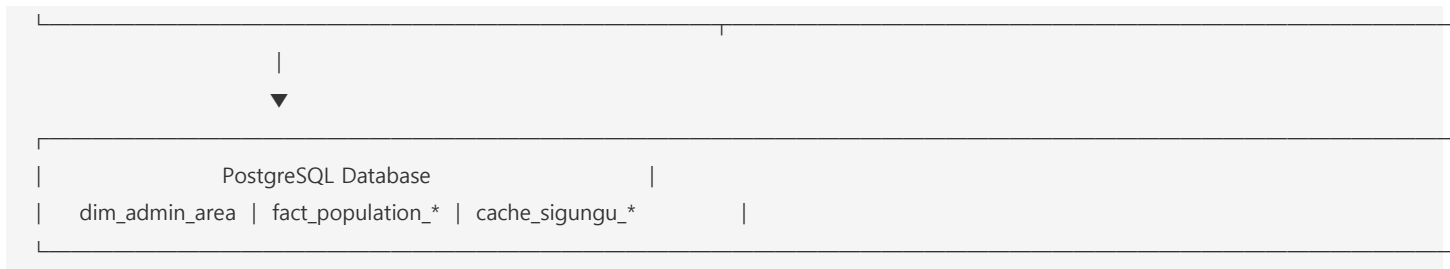
2.4 주요 지표 설명

지표명	계산식	설명
고령화율	65세+ / 전체인구 × 100	14% 이상이면 고령사회
유소년율	0~14세 / 전체인구 × 100	낮을수록 저출산
생산가능인구율	15~64세 / 전체인구 × 100	경제활동인구 비율
노령화지수	65세+ / 0~14세 × 100	100 이상이면 고령인구 > 유소년
1인가구비율	1인가구 / 전체세대 × 100	독거 가구 비율

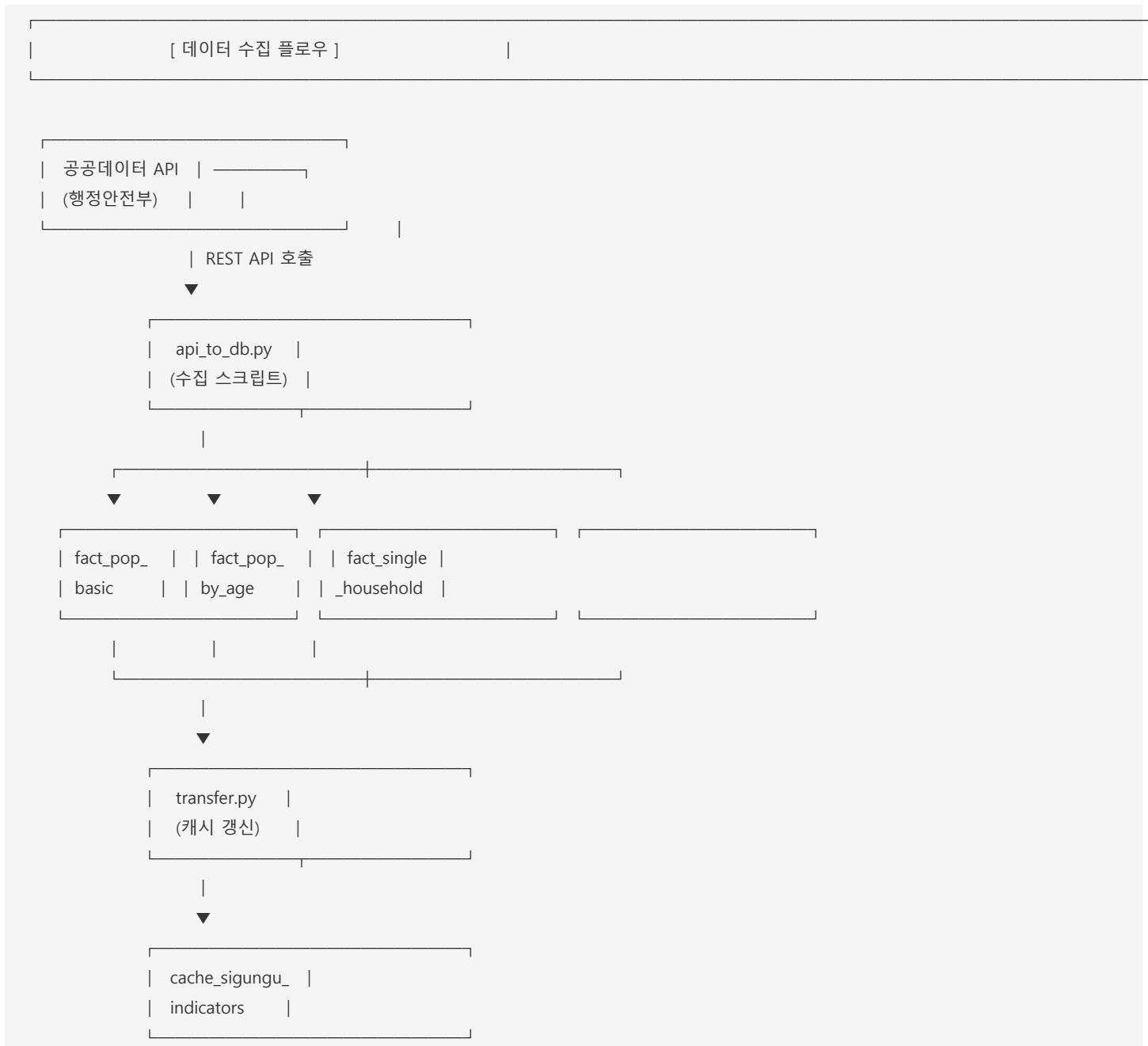
3. 시스템 아키텍처 플로우

3.1 전체 시스템 구조





3.2 데이터 수집 플로우



3.3 Text-to-SQL 질의 플로우





4. 핵심 코드 설명

4.1 디렉토리 구조

```

01_claude_project/
├── main_app.py          # Flask 메인 (진입점)
├──
├── module/              # 공통 모듈
│   ├── db.py            # DB 연결 관리
│   ├── llm_client.py     # LLM API 클라이언트
│   ├── text_to_sql.py    # Text-to-SQL 엔진
│   ├── ontology_loader.py # 온톨로지 로더
│   └── .env              # 환경변수 (비밀정보)
├──
├── 01_population/       # 인구통계 프로젝트
│   ├── api_to_db.py      # API → DB 수집
│   ├── transfer.py       # 캐시 테이블 갱신
│   ├── routes/           # Flask 라우트
│   │   ├── 데이터콩.py   # LLM 챗봇 UI
│   │   └── 대시보드1.py  # 시각화 대시보드
│   ├── ontology/         # 도메인 지식
│   │   └── database_ontology.md
│   └── templates/        # HTML 템플릿
├──
└── .venv/                # Python 가상환경
  
```

4.2 주요 파일별 역할

파일	역할	핵심 함수
`main_app.py`	Flask 서버 시작, 라우트 등록	`register_project_routes()`
`db.py`	PostgreSQL 연결	`get_db_connection()`
`llm_client.py`	Claude/OpenAI/Solar API	`LLMClient.chat()`
`text_to_sql.py`	자연어 → SQL 변환	`TextToSQL.ask()`
`api_to_db.py`	공공API 데이터 수집	`main_collect()`
`transfer.py`	캐시 테이블 갱신	`refresh_indicators()`

4.3 LLM 클라이언트 사용법

```

from module.llm_client import LLMClient, LLMClientWithFallback

# 기본 사용
llm = LLMClient(provider='claude') # claude, openai, solar 중 선택
response = llm.chat("질문 내용", system_prompt="시스템 프롬프트")

# Rate Limit 확인
rate_info = llm.get_rate_limit_info()
print(rate_info) # [claude] 요청: 900/1,000 (90.0%)

# 자동 폴백 (한도 초과 시 자동 전환)
fallback_llm = LLMClientWithFallback()
  
```



```
response = fallback_llm.chat("질문 내용")
```

4.4 Text-to-SQL 사용법

```
from module.text_to_sql import TextToSQL

t2s = TextToSQL(llm_provider='claude')
result = t2s.ask("경상북도 고령화율 상위 10개 시군구")

print(result['sql'])    # 생성된 SQL
print(result['data'])   # 결과 DataFrame
print(result['answer']) # LLM 분석 응답
```

5. 지금까지 구축한 내용

5.1 완료된 기능

[구축 완료 현황]	
데이터 수집 파이프라인	
- 공공API 4종 연동 완료	
- 월별 데이터 자동 수집 스크립트	
- 테이블 파티셔닝 (3년 단위)	
Text-to-SQL 챗봇	
- 자연어 질문 → SQL 변환	
- 근거 기반 인사이트 분석 응답	
- 다중 LLM 지원 (Claude, OpenAI, Solar)	
- Rate Limit 자동 모니터링 및 폴백	
대시보드 시각화	
- 고령화율, 유소년율 등 주요 지표 카드	
- 연령대별 인구 막대/도넛/피라미드 차트	
- 1인가구 분석 차트	
운영 인프라	
- PostgreSQL 캐시 테이블 자동 갱신	
- 온톨로지 기반 도메인 지식 관리	
- 환경변수 기반 설정 관리	

5.2 최근 업데이트 내역

날짜	내용
2026-01-10	"근거 기반 인사이트" 프롬프트 개선

2026-01-10	Rate Limit 모니터링 및 자동 폴백 기능
2026-01-10	온톨로지에 초고령(80세+, 90세+) 쿼리 예시 추가

6. 정기 운영 작업

6.1 월별 필수 작업

[월별 운영 체크리스트]	
매월 10일 이후 (전월 데이터 공개 후)	
☐ 1. 데이터 수집	
\$ cd 01_population	
\$ uv run python api_to_db.py --collect	
☐ 2. 캐시 테이블 갱신	
\$ uv run python transfer.py	
☐ 3. 데이터 검증 (챗봇에서 확인)	
질문: "최신 데이터 기준년월이 언제야?"	
→ 전월(YYYYMM) 이 나오면 정상	
☐ 4. 서버 배포 (변경사항 있을 경우)	
\$ git add . && git commit -m "월별 데이터 갱신"	
\$ git push origin main	
→ Cloudtype 자동 배포	

6.2 데이터 수집 명령어

위치: 01_population 폴더
초기화 (테이블 생성) - 최초 1회만
uv run python api_to_db.py --init
데이터 수집 - 매월 실행
uv run python api_to_db.py --collect
캐시 테이블 갱신 - 데이터 수집 후 실행
uv run python transfer.py

6.3 LLM API 사용량 확인

제공자	확인 방법
Claude	[console.anthropic.com](https:
OpenAI	[platform.openai.com](https://
Solar	[console.upstage.ai](https://c

7. Git 및 Cloudtype 배포

7.1 Git 저장소 설정

```
# 1. Git 초기화 (최초 1회)
cd C:\Users\user\01_claude_project
git init
git remote add origin https://github.com/YOUR_USERNAME/01_claude_project.git

# 2. .gitignore 설정 (중요!)
# .env 파일은 절대 커밋하지 않음
echo ".env" >> .gitignore
echo ".venv/" >> .gitignore
echo "__pycache__/" >> .gitignore
echo "*.pyc" >> .gitignore
```

7.2 Git 배포 플로우

[Git 배포 플로우]

로컬 개발 환경
(코드 수정)



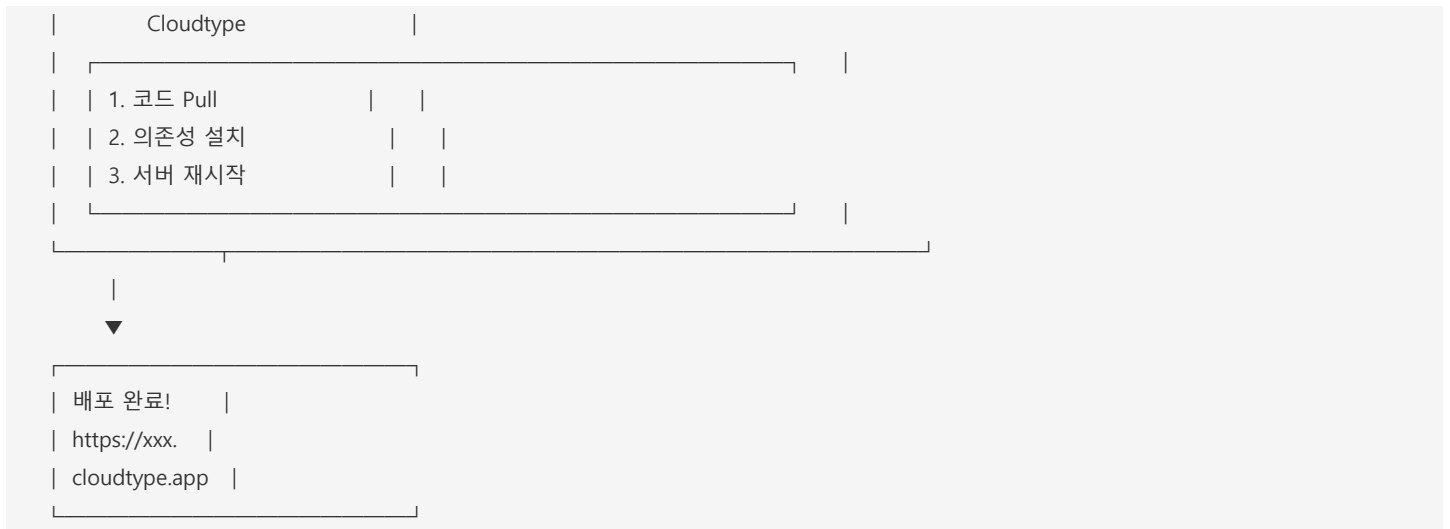
```
$ git add .
$ git commit -m "변경 내용 설명"
$ git push origin main
```



GitHub 저장소
(main 브랜치)

Webhook 트리거





7.3 Cloudtype 설정

7.3.1 프로젝트 생성

1. cloudtype.io 접속 → 로그인 2. "새 프로젝트" → GitHub 저장소 연결 3. 프레임워크: Python (Flask) 선택

7.3.2 환경변수 설정

Cloudtype 대시보드 → 프로젝트 → 설정 → 환경변수

```

DATABASE_URL=postgresql://user:password@host:port/dbname
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
UPSTAGE_API_KEY=up_...
  
```

> 중요: .env 파일은 Git에 올리지 않고, Cloudtype 환경변수로 설정

7.3.3 빌드 설정

빌드 명령어:

```
pip install -r requirements.txt
```

실행 명령어:

```
python main_app.py
```

포트: 5000

7.3.4 requirements.txt 생성

```
uv pip freeze > requirements.txt
```

```

# requirements.txt 예시
flask==3.0.0
pandas==2.1.4
psycpg2-binary==2.9.9
sqlalchemy==2.0.23
anthropic==0.18.0
openai==1.12.0
  
```

```
requests==2.31.0
python-dotenv==1.0.0
loguru==0.7.2
matplotlib==3.8.2
koreanize-matplotlib==0.1.1
numpy==1.26.3
```

7.4 배포 확인

```
# 배포 후 접속 URL
https://YOUR_PROJECT.cloudtype.app/01_population/routes/데이터콩
https://YOUR_PROJECT.cloudtype.app/01_population/routes/대시보드1
```

8. 문제 해결 가이드

8.1 자주 발생하는 오류

오류 메시지	원인	해결 방법
`relation does not exist`	테이블 미생성	`api_to_db.py --init` 실행
`ANTHROPIC_API_KEY not set`	환경변수 누락	`.env` 파일 또는 Cloudtype 환경변수 확인
`Connection refused`	DB 미실행	PostgreSQL 서버 시작
`Rate limit exceeded`	API 한도 초과	다른 LLM으로 전환 또는 대기
`SQL 생성 실패`	온톨로지 부족	`database_ontology.md`에 예시 추가

8.2 로그 확인 방법

```
# loguru 로그 설정
from loguru import logger

# 콘솔에서 실시간 확인
logger.info("정보 메시지")
logger.error("에러 메시지")

# 파일로 저장
logger.add("debug.log", level="DEBUG")
```

8.3 디버깅 체크리스트

- ☐ 1. 환경변수 확인


```
$ echo $DATABASE_URL
$ echo $ANTHROPIC_API_KEY
```
- ☐ 2. DB 연결 테스트


```
$ uv run python -c "from module.db import get_db_connection; print(get_db_connecti"
```
- ☐ 3. LLM API 테스트


```
$ uv run python -c "from module.llm_client import LLMClient; print(LLMClient().cha"
```

□ 4. 서버 실행 확인

\$ uv run python main_app.py

→ http://localhost:5000 접속

부록: 빠른 참조

A. 주요 URL

용도	URL
챗봇	`/01_population/routes/데이터콩`
대시보드	`/01_population/routes/대시보드1`

B. 주요 명령어

```
# 서버 실행
uv run python main_app.py

# 데이터 수집
uv run python 01_population/api_to_db.py --collect

# 캐시 갱신
uv run python 01_population/transfer.py

# Git 배포
git add . && git commit -m "message" && git push origin main
```

C. 환경변수 목록

```
# 필수
DATABASE_URL=postgresql://...
ANTHROPIC_API_KEY=sk-ant-...

# 선택
OPENAI_API_KEY=sk-...
UPSTAGE_API_KEY=up_...
LLM_PROVIDER=claude
LLM_AUTO_FALLBACK=true
LLM_FALLBACK_ORDER=claude,openai,solar
```

이 문서는 인구통계 대시보드 프로젝트 교육 및 인수인계를 위해 작성되었습니다. 최종 업데이트: 2026-01-11